

Build: Master Aggregator

Objective

Create a deterministic **Master Aggregator** that answers:

“What is the overall condition of this business, and what matters most right now?”

It combines outputs from:

- Viability Engine
- Survival Engine
- Stability Engine
- Growth Engine

It produces:

- Business Intelligence Score
- unified classification
- meaningful business state
- top risks
- prioritised recommendations
- confidence summary
- narrative-ready interpretation
- input for Fragility Index
- input for Scenario Simulation and ASI

Recommended Folder

`/src/lib/engines/master-aggregator`

Suggested structure:

```
/src/lib/engines/master-aggregator
├─ types.ts
```

- |— rules.ts
- |— master-aggregator.ts
- |— master-aggregator.test.ts
- |— index.ts

1. Input Contract

```
export interface MasterAggregatorInput {
  businessId: string;

  viability: {
    scores: {
      viabilityScore: number;
    };
    classification: {
      outcome: string;
      label: string;
    };
    riskFlags: RiskFlag[];
    recommendations: Recommendation[];
    confidence: Confidence;
  };

  survival: {
    scores: {
      survivalScore: number;
    };
    classification: {
      scenario: string;
      label: string;
    };
    riskFlags: RiskFlag[];
    recommendations: Recommendation[];
    confidence: Confidence;
  };

  stability: {
```

```

    scores: {
        stabilityScore: number;
    };
    classification: {
        scenario: string;
        label: string;
    };
    riskFlags: RiskFlag[];
    recommendations: Recommendation[];
    confidence: Confidence;
};

growth: {
    scores: {
        growthScore: number;
    };
    classification: {
        scenario: string;
        label: string;
    };
    riskFlags: RiskFlag[];
    recommendations: Recommendation[];
    confidence: Confidence;
};

metadata?: {
    rulesetVersion?: string;
    generatedAt?: string;
};
}

export interface RiskFlag {
    code: string;
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL" | "VERY_HIGH";
    explanation: string;
    recommendedAction: string;
    priorityScore: number;
}

```

```
export interface Recommendation {
  type: string;
  priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  text: string;
  expectedImpact?: string;
}

export interface Confidence {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
  penaltyApplied?: boolean;
}
```

2. Output Contract

```
export interface MasterAggregatorOutput {
  engine: "master_aggregator";
  version: string;
  businessId: string;

  scores: {
    viabilityScore: number;
    survivalScore: number;
    stabilityScore: number;
    growthScore: number;
    businessIntelligenceScore: number;
  };

  classification: {
    primary:
      | "HIGH_PERFORMING"
      | "STRONG_BUT_NEEDS_OPTIMISATION"
      | "AT_RISK"
      | "CRITICAL";

    structural:
      | "BALANCED_BUSINESS"
```

```

    | "VIABLE_BUT_CASH_CRITICAL"
    | "GROWTH_READY_BUT_UNSTABLE"
    | "STABLE_BUT_STAGNANT"
    | "PROFITABLE_BUT_FRAGILE"
    | "SCALING_WITH_FRAGILITY"
    | "WEAK_FOUNDATION"
    | "CRITICAL_BUSINESS_STATE";

    label: string;
};

engineSummary: {
    strongestEngine: string;
    weakestEngine: string;
    scoreSpread: number;
    balanceStatus: "BALANCED" | "UNBALANCED" | "HIGHLY_UNBALANCED";
};

topRisks: Array<RiskFlag & {
    sourceEngines: string[];
    consolidatedCode: string;
}>;

topRecommendations: Array<Recommendation & {
    sourceEngines: string[];
    reason: string;
}>;

confidence: {
    level: "LOW" | "MEDIUM" | "HIGH";
    score: number;
    note: string;
};

interpretation: {
    headline: string;
    businessStateSummary: string;
    keyStrength: string;
    keyWeakness: string;
};

```

```

    riskOutlook: string;
    strategicPriority: string;
    nextBestActions: string[];
};

trace: {
    formula: string;
    weights: {
        viability: number;
        survival: number;
        stability: number;
        growth: number;
    };
    sourceClassifications: Record<string, string>;
    rulesetVersion: string;
};
}

```

3. Master Score Formula

```

business_intelligence_score =
0.30 × viability_score +
0.25 × survival_score +
0.20 × stability_score +
0.25 × growth_score

```

4. Primary Classification

```

if businessIntelligenceScore >= 80:
    HIGH_PERFORMING

else if businessIntelligenceScore >= 60:
    STRONG_BUT_NEEDS_OPTIMISATION

else if businessIntelligenceScore >= 40:

```

```
AT_RISK
```

```
else:
```

```
    CRITICAL
```

Labels:

```
HIGH_PERFORMING = "High-Performing Business"
```

```
STRONG_BUT_NEEDS_OPTIMISATION = "Strong but Needs Optimisation"
```

```
AT_RISK = "At Risk"
```

```
CRITICAL = "Critical Business State"
```

5. Structural Classification

This is where the output becomes meaningful.

Use cross-engine logic.

A. Critical Business State

```
if businessIntelligenceScore < 40  
    or survivalScore < 30  
    or viabilityScore < 30:  
    CRITICAL_BUSINESS_STATE
```

Meaning:

The business is structurally weak and requires urgent correction before growth activity.

B. Viable but Cash Critical

```
if viabilityScore >= 70 && survivalScore < 45:  
    VIABLE_BUT_CASH_CRITICAL
```

Meaning:

The business model may work commercially, but cash survival is the immediate risk.

C. Growth Ready but Unstable

```
if growthScore >= 70 && stabilityScore < 55:  
    GROWTH_READY_BUT_UNSTABLE
```

Meaning:

The business has growth potential, but its operating structure may not support scaling safely.

D. Stable but Stagnant

```
if stabilityScore >= 70 && growthScore < 50:  
    STABLE_BUT_STAGNANT
```

Meaning:

The business may operate consistently but has limited growth momentum.

E. Profitable but Fragile

```
if viabilityScore >= 70 && stabilityScore < 55:  
    PROFITABLE_BUT_FRAGILE
```

Meaning:

The business economics look promising, but operational or customer dependency creates vulnerability.

F. Scaling with Fragility

```
if growthScore >= 70 && (survivalScore < 60 || stabilityScore < 60):  
    SCALING_WITH_FRAGILITY
```

Meaning:

The business has growth potential, but scaling may increase risk unless weak areas are corrected first.

G. Weak Foundation

```
if viabilityScore < 50 && survivalScore < 50:  
    WEAK_FOUNDATION
```

Meaning:

The business has weak commercial economics and weak short-term resilience.

H. Balanced Business

```
else:  
    BALANCED_BUSINESS
```

Meaning:

The business is broadly balanced across viability, survival, stability and growth.

6. Engine Summary Logic

Strongest Engine

highest score among viability, survival, stability, growth

Weakest Engine

lowest score among viability, survival, stability, growth

Score Spread

```
score_spread = highest_score - lowest_score
```

Balance Status

```
if scoreSpread <= 15:
    BALANCED

else if scoreSpread <= 30:
    UNBALANCED

else:
    HIGHLY_UNBALANCED
```

7. Risk Consolidation Logic

The aggregator must merge risks from all engines and avoid noise.

Step 1: Combine All Risks

```
allRisks = [
    ...viability.riskFlags,
    ...survival.riskFlags,
    ...stability.riskFlags,
    ...growth.riskFlags
]
```

Step 2: Consolidate Similar Risks

Create mapping:

```

const consolidatedRiskMap = {
  LOW_MARGIN_RISK: "COMMERCIAL_ECONOMICS_RISK",
  NO_BREAK_EVEN_RISK: "COMMERCIAL_ECONOMICS_RISK",
  SLOW_PAYBACK_RISK: "COMMERCIAL_ECONOMICS_RISK",

  CRITICAL_RUNWAY_RISK: "FINANCIAL_SURVIVAL_RISK",
  LOW_RUNWAY_RISK: "FINANCIAL_SURVIVAL_RISK",
  HIGH_BURN_RATE: "FINANCIAL_SURVIVAL_RISK",
  NEGATIVE_CASHFLOW: "FINANCIAL_SURVIVAL_RISK",

  CUSTOMER_CONCENTRATION_RISK: "CUSTOMER_DEPENDENCY_RISK",
  SINGLE_CUSTOMER_DEPENDENCY: "CUSTOMER_DEPENDENCY_RISK",
  TOP3_CUSTOMER_CONCENTRATION: "CUSTOMER_DEPENDENCY_RISK",

  KEY_PERSON_RISK: "OPERATIONAL_DEPENDENCY_RISK",
  CAPACITY_CONSTRAINT: "OPERATIONAL_CAPACITY_RISK",
  CAPACITY_CONSTRAINT_RISK: "OPERATIONAL_CAPACITY_RISK",
  CAPACITY_OVERLOAD: "OPERATIONAL_CAPACITY_RISK",

  WEAK_PRICING_POWER: "MARKET_POSITIONING_RISK",
  COMMODITY_RISK: "MARKET_POSITIONING_RISK",
  COMPETITIVE_PRESSURE: "MARKET_POSITIONING_RISK",

  LOW_SCALABILITY: "SCALABILITY_RISK",
  EXPANSION_LIMITED: "SCALABILITY_RISK"
};

```

Step 3: Priority Score

Calculate consolidated priority:

```

consolidated_priority =
max(priorityScore of grouped risks)
+ severity_bonus
+ multi_engine_bonus

```

Severity bonus:

CRITICAL / VERY_HIGH = 30

HIGH = 20

MEDIUM = 10

LOW = 0

Multi-engine bonus:

if same consolidated risk appears across 2+ engines:
+10

Step 4: Return Top 5 Risks

Sort by:

1. severity
2. consolidated priority
3. number of source engines

Return maximum 5.

8. Recommendation Consolidation

Step 1: Combine Recommendations

```
allRecommendations = [  
    ...viability.recommendations,  
    ...survival.recommendations,  
    ...stability.recommendations,  
    ...growth.recommendations  
]
```

Step 2: Conflict Resolution

Important examples:

Hiring Conflict

```
if growth recommends hiring
AND survivalScore < 50:
    replace with:
        "Delay hiring until runway improves or use flexible contractors."
```

Growth Conflict

```
if growthScore high
AND stabilityScore low:
    prioritize:
        "Fix operational stability before scaling aggressively."
```

Pricing Conflict

```
if viability recommends price increase
AND demand evidence weak:
    recommend:
        "Test price increase with a small customer segment before full
        rollout."
```

Step 3: Prioritise Top 5

Ranking formula:

```
recommendation_priority =
business_impact
+ urgency
+ risk_reduction_value
+ confidence_bonus
- implementation_difficulty_penalty
```

Return top 5.

9. Confidence Aggregation

Convert confidence level to score:

HIGH = 90

MEDIUM = 65

LOW = 35

Overall confidence:

```
confidence_score =  
average(viability_confidence, survival_confidence,  
stability_confidence, growth_confidence)
```

Penalty:

```
if any engine confidence LOW:  
    subtract 5
```

Classification:

```
if confidenceScore >= 80 => HIGH  
else if confidenceScore >= 50 => MEDIUM  
else => LOW
```

10. Meaningful Interpretation Logic

The output should not just say “Score is 65”.

It must say something like:

“The business has strong growth potential, but survival and stability weaknesses mean it should not scale aggressively yet.”

Headline Rules

High Performing

The business is performing strongly across core intelligence dimensions.

Strong but Needs Optimisation

The business is generally strong, but specific weaknesses need correction before aggressive growth.

At Risk

The business shows meaningful risk and needs focused corrective action.

Critical

The business is in a critical state and should prioritise survival and structural correction.

11. Strategic Priority Logic

Determine priority based on weakest engine.

```
if weakestEngine === "survival":  
    "Protect cash runway and reduce burn."
```

```
if weakestEngine === "viability":  
    "Improve business model economics and demand validation."
```

```
if weakestEngine === "stability":  
    "Reduce structural fragility and operational dependency."
```

```
if weakestEngine === "growth":  
    "Improve scalability, differentiation and market readiness."
```

12. Output Example

```
{
  "engine": "master_aggregator",
  "version": "1.0.0",
  "businessId": "bus_001",
  "scores": {
    "viabilityScore": 78,
    "survivalScore": 42,
    "stabilityScore": 55,
    "growthScore": 81,
    "businessIntelligenceScore": 65.35
  },
  "classification": {
    "primary": "STRONG_BUT_NEEDS_OPTIMISATION",
    "structural": "SCALING_WITH_FRAGILITY",
    "label": "Strong but Scaling with Fragility"
  },
  "engineSummary": {
    "strongestEngine": "growth",
    "weakestEngine": "survival",
    "scoreSpread": 39,
    "balanceStatus": "HIGHLY_UNBALANCED"
  },
  "topRisks": [
    {
      "consolidatedCode": "FINANCIAL_SURVIVAL_RISK",
      "code": "LOW_RUNWAY_RISK",
      "severity": "HIGH",
      "sourceEngines": ["survival"],
      "explanation": "The business has limited cash runway.",
      "recommendedAction": "Reduce burn and improve short-term cash
inflows.",
      "priorityScore": 88
    }
  ],
}
```



```

"topRecommendations": [
  {
    "type": "cash_preservation",
    "priority": "HIGH",
    "text": "Improve cash runway before scaling aggressively.",
    "sourceEngines": ["survival", "growth"],
    "reason": "Growth potential is strong, but survival weakness
creates scaling risk."
  }
],
"confidence": {
  "level": "MEDIUM",
  "score": 66,
  "note": "This result is based on mixed confidence data across the
four engines."
},
"interpretation": {
  "headline": "The business is promising but exposed.",
  "businessStateSummary": "The business has strong viability and
growth potential, but weak survival resilience means scaling should be
controlled.",
  "keyStrength": "Growth potential is the strongest area.",
  "keyWeakness": "Cash survival is the weakest area.",
  "riskOutlook": "Scaling before improving runway could increase
financial pressure.",
  "strategicPriority": "Protect cash runway and reduce burn.",
  "nextBestActions": [
    "Improve short-term cash position.",
    "Delay major fixed-cost expansion.",
    "Run a cost-reduction or pricing scenario before scaling."
  ]
}
}

```

13. Unit Tests

Create tests for:

High Performing Balanced Case

Expected:

- HIGH_PERFORMING
- BALANCED_BUSINESS
- balance status BALANCED

Viable but Cash Critical

Input:

- viability ≥ 70
- survival < 45

Expected:

- VIABLE_BUT_CASH_CRITICAL
- strategic priority cash runway

Scaling with Fragility

Input:

- growth ≥ 70
- survival or stability < 60

Expected:

- SCALING_WITH_FRAGILITY

Stable but Stagnant

Input:

- stability ≥ 70
- growth < 50

Expected:

- STABLE_BUT_STAGNANT

Weak Foundation

Input:

- viability < 50
- survival < 50

Expected:

- WEAK_FOUNDATION

Risk Consolidation

Input:

- LOW_RUNWAY_RISK + HIGH_BURN_RATE

Expected:

- consolidated risk FINANCIAL_SURVIVAL_RISK

Recommendation Conflict

Input:

- growth recommends hiring
- survival score < 50

Expected:

- recommendation changed to delay hiring/flexible contractor

14. Acceptance Criteria

Codex should complete this when:

- Master Aggregator accepts the defined input contract
- It calculates master score correctly
- It classifies primary state correctly
- It classifies structural state meaningfully
- It identifies strongest and weakest engine
- It consolidates risks
- It resolves recommendation conflicts
- It returns top risks and recommendations
- It produces meaningful interpretation
- It includes confidence aggregation
- Unit tests pass
- No database calls inside aggregator
- No LLM calls inside aggregator

15. Codex Prompt

Build the EnterprateAI Master Aggregator in `/src/lib/engines/master-aggregator`.

The aggregator must be a pure deterministic TypeScript module. It must not call the database, API, or LLM.

It should accept `MasterAggregatorInput` containing outputs from:

- viability engine
- survival engine
- stability engine
- growth engine

It should return `MasterAggregatorOutput`.

Implement:

- business intelligence score calculation
- primary classification
- structural classification
- strongest engine detection
- weakest engine detection

- score spread
- balance status
- risk consolidation
- recommendation consolidation
- recommendation conflict resolution
- confidence aggregation
- meaningful interpretation
- trace object

Use this formula:

```
businessIntelligenceScore =  
0.30 * viabilityScore +  
0.25 * survivalScore +  
0.20 * stabilityScore +  
0.25 * growthScore
```

Create:

- types.ts
- rules.ts
- master-aggregator.ts
- master-aggregator.test.ts
- index.ts

Add tests for:

- high performing balanced case
- viable but cash critical
- scaling with fragility
- stable but stagnant
- weak foundation
- risk consolidation
- recommendation conflict resolution

Export the aggregator from index.ts.

Next: Fragility Index Reads risks + scores + dependencies.

Build: Fragility Index

Objective

Create a deterministic **Fragility Index Engine** that answers:

“How easily can this business break under pressure?”

It reads:

- master score
- engine scores
- risk flags
- customer concentration risks
- cash survival risks
- operational dependencies
- capacity risks
- market/growth risks

It outputs:

- fragility index
- fragility classification
- fragility dimensions
- top fragility factors
- propagation risks
- mitigation recommendations
- ASI-ready triggers

Recommended Folder

`/src/lib/engines/fragility`

Suggested structure:

```
/src/lib/engines/fragility
├─ types.ts
├─ rules.ts
├─ fragility-engine.ts
├─ fragility-engine.test.ts
└─ index.ts
```

1. Input Contract

```
export interface FragilityEngineInput {
  businessId: string;

  scores: {
    viabilityScore: number;
    survivalScore: number;
    stabilityScore: number;
    growthScore: number;
    businessIntelligenceScore: number;
  };

  risks: Array<{
    code: string;
    consolidatedCode?: string;
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL" | "VERY_HIGH";
    sourceEngines?: string[];
    priorityScore: number;
  }>;

  dependencies?: Array<{
    dependencyType:
      | "key_person"
      | "customer"
      | "supplier"
      | "system"
      | "process"
      | "channel";
```

```

    criticalityLevel: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    active: boolean;
  }>;

  metrics?: {
    runwayMonths?: number | null;
    burnRatePctOfRevenue?: number | null;
    top1CustomerRevenuePct?: number | null;
    top3CustomersRevenuePct?: number | null;
    revenueVolatilityRatio?: number | null;
    capacityUtilisationPct?: number | null;
    backlogVolume?: number | null;
    pricingPowerScore?: number | null;
    competitiveScore?: number | null;
  };

  metadata?: {
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
  };
}

```

2. Output Contract

```

export interface FragilityEngineOutput {
  engine: "fragility";
  version: string;
  businessId: string;

  fragility: {
    index: number;
    classification:
      | "RESILIENT"
      | "STABLE_WITH_EXPOSURE"
      | "FRAGILE"
      | "HIGHLY_FRAGILE"
      | "CRITICAL_FRAGILITY";
  };
}

```



```
    label: string;
};

dimensions: {
    cashFragility: number;
    revenueFragility: number;
    customerFragility: number;
    operationalFragility: number;
    capacityFragility: number;
    marketFragility: number;
};

topFragilityFactors: Array<{
    factorCode: string;
    dimension:
        | "cash"
        | "revenue"
        | "customer"
        | "operational"
        | "capacity"
        | "market";
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    scoreImpact: number;
    explanation: string;
    mitigationAction: string;
}>;

propagationRisks: Array<{
    source: string;
    path: string[];
    propagationScore: number;
    explanation: string;
}>;

recommendations: Array<{
    type: string;
    priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    text: string;
```

```

    expectedFragilityReduction?: number;
  }>;

  asiTriggers: Array<{
    triggerCode: string;
    triggerType: "risk" | "opportunity" | "trend";
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    reason: string;
  }>;

  confidence: {
    level: "LOW" | "MEDIUM" | "HIGH";
    note: string;
  };

  interpretation: {
    headline: string;
    summary: string;
    weakestDimension: string;
    riskOutlook: string;
    strategicPriority: string;
  };

  trace: Record<string, unknown>;
}

```

3. Fragility Dimensions

Calculate six dimensions from scores, risks, metrics and dependencies.

All dimension scores are **0–100**, where:

0 = resilient

100 = extremely fragile

4. Cash Fragility

Inputs

- survivalScore
- runwayMonths
- burnRatePctOfRevenue
- FINANCIAL_SURVIVAL_RISK
- LOW_RUNWAY_RISK
- CRITICAL_RUNWAY_RISK
- HIGH_BURN_RATE

Base Rule

`cashFragility = 100 - survivalScore`

Adjustments

```
if runwayMonths !== null && runwayMonths < 3:  
    cashFragility += 25
```

```
else if runwayMonths !== null && runwayMonths < 6:  
    cashFragility += 15
```

```
if burnRatePctOfRevenue !== null && burnRatePctOfRevenue > 0.5:  
    cashFragility += 15
```

```
if risk FINANCIAL_SURVIVAL_RISK exists:  
    cashFragility += 10
```

Cap:

```
cashFragility = min(max(cashFragility, 0), 100)
```

5. Revenue Fragility

Inputs

- viabilityScore
- revenueVolatilityRatio
- REVENUE_UNSTABLE
- UNPROVEN_DEMAND_RISK
- COMMERCIAL_ECONOMICS_RISK

Base Rule

`revenueFragility = 100 - viabilityScore`

Adjustments

```
if revenueVolatilityRatio !== null && revenueVolatilityRatio > 1:  
    revenueFragility += 15  
if risk REVENUE_UNSTABLE exists:  
    revenueFragility += 15  
if risk UNPROVEN_DEMAND_RISK exists:  
    revenueFragility += 15
```

Cap 0–100.

6. Customer Fragility

Inputs

- top1CustomerRevenuePct
- top3CustomersRevenuePct
- CUSTOMER_DEPENDENCY_RISK
- SINGLE_CUSTOMER_DEPENDENCY

- customer dependencies

Base Rule

```
if top1CustomerRevenuePct === null:
    customerFragility = 50

else if top1CustomerRevenuePct > 0.6:
    customerFragility = 95

else if top1CustomerRevenuePct > 0.4:
    customerFragility = 75

else if top1CustomerRevenuePct > 0.2:
    customerFragility = 45

else:
    customerFragility = 15
```

Adjustments

```
if top3CustomersRevenuePct !== null && top3CustomersRevenuePct > 0.7:
    customerFragility += 10
if risk CUSTOMER_DEPENDENCY_RISK exists:
    customerFragility += 10
if active customer dependency with HIGH/CRITICAL criticality exists:
    customerFragility += 10
```

Cap 0–100.

7. Operational Fragility

Inputs

- stabilityScore

- dependencies
- KEY_PERSON_RISK
- OPERATIONAL_DEPENDENCY_RISK
- OPERATIONAL_SINGLE_POINT_FAILURE

Base Rule

`operationalFragility = 100 - stabilityScore`

Adjustments

```
if risk KEY_PERSON_RISK exists:
    operationalFragility += 15
if risk OPERATIONAL_SINGLE_POINT_FAILURE exists:
    operationalFragility += 25
for each active dependency with HIGH criticality:
    operationalFragility += 5

for each active dependency with CRITICAL criticality:
    operationalFragility += 10
```

Cap 0–100.

8. Capacity Fragility

Inputs

- capacityUtilisationPct
- backlogVolume
- CAPACITY_CONSTRAINT
- CAPACITY_OVERLOAD
- OPERATIONAL_CAPACITY_RISK

Base Rule

```
if capacityUtilisationPct === null:
    capacityFragility = 50

else if capacityUtilisationPct > 0.9:
    capacityFragility = 85

else if capacityUtilisationPct > 0.75:
    capacityFragility = 55

else if capacityUtilisationPct < 0.5:
    capacityFragility = 45

else:
    capacityFragility = 20
```

Adjustments

```
if backlogVolume !== null && backlogVolume > 0 &&
capacityUtilisationPct > 0.9:
    capacityFragility += 10
if risk OPERATIONAL_CAPACITY_RISK exists:
    capacityFragility += 10
```

Cap 0–100.

9. Market Fragility

Inputs

- growthScore
- pricingPowerScore
- competitiveScore
- MARKET_POSITIONING_RISK

- COMMODITY_RISK
- SCALABILITY_RISK

Base Rule

`marketFragility = 100 - growthScore`

Adjustments

```
if pricingPowerScore !== null && pricingPowerScore <= 40:
    marketFragility += 15
if competitiveScore !== null && competitiveScore <= 40:
    marketFragility += 15
if risk COMMODITY_RISK exists:
    marketFragility += 25
if risk SCALABILITY_RISK exists:
    marketFragility += 10
```

Cap 0–100.

10. Final Fragility Index Formula

```
fragility_index =
0.30 × cash_fragility +
0.15 × revenue_fragility +
0.20 × customer_fragility +
0.20 × operational_fragility +
0.15 × capacity_fragility +
0.15 × market_fragility
```

Important: this totals **1.15**, so normalise by total weight.

Implementation:

```
const totalWeight = 1.15;
```



```
fragilityIndex =  
weightedSum / totalWeight;
```

Alternative cleaner formula:

```
fragility_index =  
0.25 × cash_fragility +  
0.15 × revenue_fragility +  
0.20 × customer_fragility +  
0.20 × operational_fragility +  
0.10 × capacity_fragility +  
0.10 × market_fragility
```

Use this cleaner formula for MVP.

11. Recommended MVP Formula

```
fragility_index =  
0.25 × cash_fragility +  
0.15 × revenue_fragility +  
0.20 × customer_fragility +  
0.20 × operational_fragility +  
0.10 × capacity_fragility +  
0.10 × market_fragility
```

12. Fragility Classification

```
if fragilityIndex <= 20:  
    RESILIENT  
  
else if fragilityIndex <= 40:  
    STABLE_WITH_EXPOSURE  
  
else if fragilityIndex <= 60:  
    FRAGILE
```

```
else if fragilityIndex <= 80:  
    HIGHLY_FRAGILE
```

```
else:  
    CRITICAL_FRAGILITY
```

Labels:

```
RESILIENT = "Resilient"  
STABLE_WITH_EXPOSURE = "Stable with Exposure"  
FRAGILE = "Fragile"  
HIGHLY_FRAGILE = "Highly Fragile"  
CRITICAL_FRAGILITY = "Critical Fragility"
```

13. Top Fragility Factors

Generate factors when dimension exceeds thresholds.

Cash

```
if cashFragility >= 60:  
    CASH_FRAGILITY
```

Customer

```
if customerFragility >= 60:  
    CUSTOMER_FRAGILITY
```

Operational

```
if operationalFragility >= 60:  
    OPERATIONAL_FRAGILITY
```

Capacity

```
if capacityFragility >= 60:  
    CAPACITY_FRAGILITY
```

Market

```
if marketFragility >= 60:  
    MARKET_FRAGILITY
```

Return top 5 by score impact.

14. Propagation Risk Logic

Create deterministic propagation paths.

Customer Dependency Path

Trigger:

```
customerFragility >= 70
```

Path:

```
Top customer dependency  
→ revenue loss exposure  
→ cashflow pressure  
→ survival risk  
→ higher business fragility
```

Cash Pressure Path

Trigger:

```
cashFragility >= 70
```

Path:

Low cash runway

- reduced decision flexibility
- delayed growth investment
- higher survival risk

Operational Dependency Path

Trigger:

`operationalFragility >= 70`

Path:

Key person or process dependency

- delivery disruption
- customer dissatisfaction
- revenue instability
- cash pressure

Capacity Path

Trigger:

`capacityFragility >= 70`

Path:

Capacity overload

- delivery delays
- customer churn risk
- revenue instability

Market Path

Trigger:

`marketFragility >= 70`

Path:

Weak positioning

→ pricing pressure

→ margin compression

→ commercial fragility

15. Recommendations Logic

Generate top recommendations based on weakest dimensions.

Cash Fragility

Improve runway by reducing burn, increasing short-term inflows, or delaying fixed commitments.

Customer Fragility

Reduce dependency on top customers by building alternative revenue sources.

Operational Fragility

Document key processes, create backups, and reduce key-person dependency.

Capacity Fragility

Improve delivery capacity or reduce backlog before scaling demand.

Market Fragility

Improve differentiation and pricing power before expanding aggressively.

Return top 5.

16. ASI Triggers

Generate ASI triggers from fragility.

HIGH_FRAGILITY_TRIGGER

```
if fragilityIndex > 60
```

CRITICAL_FRAGILITY_TRIGGER

```
if fragilityIndex > 80
```

CASH_FRAGILITY_TRIGGER

```
if cashFragility > 70
```

CUSTOMER_FRAGILITY_TRIGGER

```
if customerFragility > 70
```

OPERATIONAL_FRAGILITY_TRIGGER

```
if operationalFragility > 70
```

Each trigger should include:

```
{
  triggerCode: "CASH_FRAGILITY_TRIGGER",
  triggerType: "risk",
  severity: "HIGH",
  reason: "Cash fragility is above safe threshold."
}
```

17. Interpretation Logic

RESILIENT

The business appears structurally resilient and able to absorb normal operating pressure.

STABLE_WITH_EXPOSURE

The business is broadly stable but has some exposure that should be monitored.

FRAGILE

The business has meaningful structural vulnerabilities that could become harmful under pressure.

HIGHLY_FRAGILE

The business is highly exposed and should correct key weaknesses before scaling.

CRITICAL_FRAGILITY

The business is critically fragile and requires immediate corrective action.

Strategic priority should be based on the highest fragility dimension.

18. Confidence Logic

```
if metadata.confidenceLevel exists:  
    use it
```

```
else if risks.length > 0 && metrics.hasRunway + customer
concentration + capacity:
    MEDIUM
```

```
else:
    LOW
```

19. Unit Tests

Create tests for:

Resilient Case

- high scores
- low risks
- low dependency
- Expected:
- RESILIENT

Cash Fragility Case

- survival score low
- runway < 3
- Expected:
- high cashFragility
- CASH_FRAGILITY_TRIGGER

Customer Fragility Case

- top customer > 60%
- Expected:
- high customerFragility
- customer propagation risk

Operational Fragility Case

- critical dependency
- key person risk
- Expected:
- operational fragility high

Capacity Fragility Case

- utilisation > 90%
- backlog > 0
- Expected:
- capacity fragility high

Market Fragility Case

- growth low
- commodity risk
- Expected:
- market fragility high

Highly Fragile Case

- multiple high dimensions
- Expected:
- HIGHLY_FRAGILE or CRITICAL_FRAGILITY

20. Acceptance Criteria

Codex should complete this when:

- Fragility Engine accepts the defined input contract
- It calculates all six fragility dimensions
- It calculates final fragility index

- It classifies fragility correctly
- It identifies top fragility factors
- It generates propagation risks
- It generates mitigation recommendations
- It generates ASI triggers
- It includes confidence and interpretation
- Unit tests pass
- No database calls inside engine
- No LLM calls inside engine

21. Codex Prompt

Build the EnterprateAI Fragility Index Engine in
/src/lib/engines/fragility.

The engine must be a pure deterministic TypeScript module. It must not call the database, API, or LLM.

It should accept FragilityEngineInput and return FragilityEngineOutput.

Implement:

- cash fragility
- revenue fragility
- customer fragility
- operational fragility
- capacity fragility
- market fragility
- final fragility index
- fragility classification
- top fragility factors
- propagation risks
- recommendations
- ASI triggers
- confidence layer
- interpretation

- trace object

Use this MVP formula:

```
fragilityIndex =  
0.25 * cashFragility +  
0.15 * revenueFragility +  
0.20 * customerFragility +  
0.20 * operationalFragility +  
0.10 * capacityFragility +  
0.10 * marketFragility
```

Create:

- types.ts
- rules.ts
- fragility-engine.ts
- fragility-engine.test.ts
- index.ts

Add tests for:

- resilient case
- cash fragility case
- customer fragility case
- operational fragility case
- capacity fragility case
- market fragility case
- highly fragile case

Export the engine from index.ts.